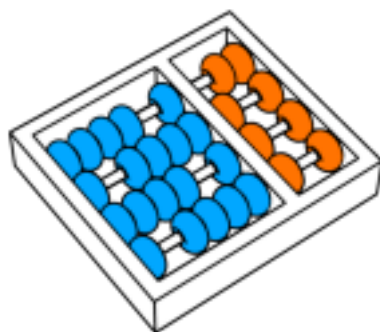




Laboratório 2

MC714 - 2026.1S

George Sá - RA: 231529
Gabriel Jerônimo - RA: 247112

1.Introdução:

Este relatório documenta a implementação de um conjunto de algoritmos fundamentais para a coordenação e sincronização em sistemas distribuídos.

O desenvolvimento de sistemas distribuídos traz desafios relacionados à ausência de um relógio físico global e de memória compartilhada, exigindo mecanismos robustos para ordenação de eventos, controle de concorrência e tolerância a falhas. Para endereçar esses problemas e demonstrar seu funcionamento na prática, este projeto engloba a implementação de três algoritmos:

1. Relógio Lógico de Lamport: mecanismo para a ordenação parcial de eventos no sistema, permitindo que os processos distribuídos concordem sobre a sequência em que as operações ocorrem.

2. Algoritmo de Ricart-Agrawala: uma solução distribuída para o problema de exclusão mútua, baseada em timestamps e permissões, garantindo que múltiplos processos não acessem simultaneamente uma região crítica.

3. Algoritmo Bully: um algoritmo de eleição de líder, responsável por coordenar a escolha de um novo coordenador, caso o líder atual venha a falhar.

A solução foi desenvolvida na linguagem Go, por sua facilidade nativa para a concorrência. Além disso, a comunicação entre os nós distribuídos foi construída do zero, ocorrendo através da troca de mensagens no formato JSON utilizando sockets sobre o protocolo TCP.

Por fim, para simular um ambiente distribuído realista e facilitar a replicação, o sistema foi orquestrado utilizando contêineres Docker. Nas seções seguintes, serão detalhadas as decisões arquiteturais, o funcionamento de cada algoritmo, a infraestrutura de rede, bem como os cenários de testes e os resultados dos experimentos realizados para validar a solução.

2. Arquitetura e Comunicação:

O sistema foi projetado com uma arquitetura descentralizada e P2P, onde não existe um servidor central de roteamento. Todos os processos/nós executam o mesmo binário base e possuem responsabilidades simétricas no que diz respeito à comunicação. Cada nó é instanciado conhecendo seu próprio identificador (nodeID) e a quantidade total de nós na rede (totalNodes).

A infraestrutura de rede foi implementada do zero na linguagem Go, dispensando o uso de frameworks complexos. Dessa forma, o objetivo foi lidar diretamente com os desafios de comunicação em baixo nível. Os principais pilares dessa camada são:

2.1 Protocolo e Formato de Mensagens

Toda a troca de dados entre os processos ocorre sobre sockets TCP. Para garantir flexibilidade e fácil depuração, os dados trafegados são serializados no formato JSON. Foi padronizada uma estrutura única de mensagem (Message em *message.go*) para todos os algoritmos, contendo:

- *Type*: O tipo da mensagem (ex: "REQUEST", "REPLY" para exclusão mútua; "ELECTION", "HEARTBEAT" para o Bully).
- *SenderID*: A identificação de qual nó originou a requisição.
- *Timestamp*: O valor do Relógio Lógico de Lamport no momento do envio.
- *Data*: Um campo opcional para cargas úteis específicas.

```
type Message struct {  
    Type      string `json:"type"`  
    SenderID  int    `json:"sender_id"`  
    Timestamp int    `json:"timestamp"`  
    Data      string `json:"data,omitempty"`  
}
```

2.2 Roteamento Baseado em Eventos (Event-Driven)

Para integrar a rede com os algoritmos lógicos, foi implementado um padrão de callbacks. A estrutura *Transport* permite que os algoritmos implementados no projeto registrem funções para lidar com mensagens específicas através de chamadas como *transport.On("ELECTION", handler)*.

Quando o servidor TCP recebe e decodifica um JSON, ele lê o campo *Type* e despacha a mensagem imediatamente para o tratador correto. Essa abordagem assíncrona garante que a chegada de uma mensagem de eleição não bloqueie o processamento de um pedido de seção crítica, por exemplo.

2.3 Topologia e Endereçamento

A resolução de endereços tira proveito do DNS interno do Docker. Como a especificação demandava a execução em múltiplos hosts ou contêineres, o envio de mensagens constrói o endereço de destino dinamicamente no formato *node{ID}:5000*. Se a conexão TCP falha (como no caso de um nó inativo), a rede retorna um erro de timeout (2 segundos), o que é importante para o algoritmo Bully detectar indisponibilidade durante as eleições. O módulo de rede também provê abstrações de alto nível, como *Broadcast()* (envio para todos) e *SendToHigherNodes()* (usado na eleição).

3. Algoritmos implementados

3.1 Relógio lógico de Lamport

Em um sistema distribuído sem memória compartilhada, não podemos confiar em relógios físicos devido à latência da rede e à falta de sincronização perfeita entre as máquinas. Para contornar esse problema e permitir que todos os processos cheguem a um consenso sobre a ordem dos eventos, implementamos o Relógio Lógico de Lamport.

A estrutura `LamportClock` mantém um contador inteiro simples que dita a noção de tempo daquele nó específico. O funcionamento deste relógio segue duas regras básicas estipuladas na literatura, que foram implementadas através de funções distintas no código:

Eventos Locais e de Envio: Antes de executar qualquer evento de relevância, como decidir solicitar acesso à seção crítica ou enviar uma mensagem para a rede, o processo invoca as funções `Tick()` ou `SendTick()`. Ambas incrementam o contador local em uma unidade. O valor resultante é então anexado em todas as mensagens que saem do nó via formato JSON.

```
func (lc *LamportClock) Tick() int {
    lc.mu.Lock()
    defer lc.mu.Unlock()
    lc.time++
    return lc.time
}
```

```
func (lc *LamportClock) SendTick() int {
    lc.mu.Lock()
    defer lc.mu.Unlock()
    lc.time++
    return lc.time
}
```

Eventos de Recebimento: Quando o processo recebe uma mensagem de outro nó, ele analisa o timestamp embutido nela e invoca a função `ReceiveTick`. O relógio local é então atualizado para assumir o maior valor entre o seu próprio tempo atual e o tempo recebido na mensagem, sendo posteriormente incrementado em uma unidade. Essa ação matemática garante que a recepção de uma mensagem seja registrada como um evento que aconteceu rigorosamente depois do envio daquela mesma mensagem.

```
func (lc *LamportClock) ReceiveTick(receivedTimestamp int) int {
    lc.mu.Lock()
    defer lc.mu.Unlock()
    if receivedTimestamp > lc.time {
        lc.time = receivedTimestamp
    }
    lc.time++
    return lc.time
}
```

3.2 Ricart-Agrawala

O controle de concorrência distribuído foi garantido pela implementação do Algoritmo de Ricart-Agrawala. Este algoritmo, diferentemente de abordagens centralizadas baseadas em um único coordenador de travas, exige que todos os processos participem ativamente da tomada de decisão. A lógica funciona na troca de mensagens e na ordem temporal representada pelo Relógio Lógico de Lamport.

Na nossa arquitetura, a estrutura *RicartAgrawala* mantém o estado atual do nó em relação ao recurso compartilhado, podendo assumir três valores: *Released* (Liberado), *Wanted* (Desejado) ou *Held* (na Seção Crítica). O fluxo de concessão ocorre da seguinte forma:

Solicitação de Acesso: Quando um nó deseja entrar na Seção Crítica, ele altera seu estado para *Wanted*, salva o valor atual do seu relógio de Lamport como o *requestTimestamp* daquela operação e transmite uma mensagem do tipo REQUEST em broadcast para todos os outros nós ativos. O nó então bloqueia seu fluxo principal aguardando receber as respostas de todos os pares.

Processamento de Requisição: Quando qualquer nó recebe um REQUEST, ele avalia seu próprio estado e o timestamp embutido na mensagem para tomar uma decisão. Se o nó estiver no estado Held, ele enfileira a requisição em uma lista de espera (*deferredReplies*).

Se o nó estiver no estado Wanted, ele compara os timestamps. O nó que tiver o timestamp menor ganha a prioridade. Caso os timestamps sejam idênticos, o impasse é resolvido usando o ID do nó como fator de desempate, dando prioridade ao nó de menor ID. Se o nó local tiver menor prioridade, ele enfileira a requisição. Caso contrário, ele responde imediatamente com um REPLY.

Se o nó estiver no estado Released, ele responde imediatamente com um REPLY.

Liberação do Recurso: Assim que o nó reúne os REPLYs de todos os participantes aos quais enviou a solicitação, ele entende que tem a permissão unânime, muda seu estado para Held e executa as operações na Seção Crítica. Ao finalizar, ele volta o estado para Released e percorre a sua lista de requisições enfileiradas, enviando agora os REPLYs atrasados para destravar os nós que estavam aguardando a sua vez.

Assim, esse mecanismo garante que não haja a ocorrência de deadlocks ou starvation, já que a ordenação temporal estrita imposta pelo Relógio de Lamport cria uma fila de prioridade justa e globalmente aceita em todo o sistema.

3.3 Algoritmo de Eleição de Líder (Bully)

A escolha e manutenção de um processo coordenador único no sistema foi implementada através do Algoritmo Bully. A premissa fundamental desse algoritmo é que, em caso de falha do líder atual, o processo ativo com o maior Identificador (ID) sempre vence a nova eleição e assume a liderança.

Na nossa solução, a dinâmica de detecção de falhas e o processo de eleição foram construídos com base em um sistema de Heartbeats e timeouts. O ciclo de vida da liderança ocorre nas seguintes etapas:

Monitoramento e Detecção de Falha: O nó que possui a liderança tem a responsabilidade de emitir mensagens do tipo HEARTBEAT em broadcast a cada 2 segundos. Enquanto isso, os nós subordinados executam uma rotina paralela (*MonitorLeader*) que verifica constantemente se o último pulso foi recebido dentro de uma janela de tempo segura. Caso o tempo exceda o limite de tolerância, o nó assume que o líder falhou ou foi desconectado da rede.

Desencadeamento da Eleição: Ao detectar a falha, o nó muda seu estado interno para indicar que uma eleição está em andamento e invoca a função *StartElection()*. Ele então envia uma mensagem ELECTION exclusivamente para os nós com identificadores maiores que o seu. Se um nó receptor tiver um ID maior, ele intercepta o pedido, responde imediatamente com uma mensagem de OK e inicia o seu próprio processo de eleição.

Declaração de Vitória: Se um nó dispara as mensagens de ELECTION para IDs maiores e não recebe nenhum OK dentro do tempo limite, isso significa que ele é atualmente o nó de maior ID operante na rede. Nesse cenário, o nó invoca a função *declareVictory()*, autodeclarando-se o novo líder.

Confirmação: Imediatamente após vencer, o novo líder transmite uma mensagem COORDINATOR para todos os outros nós ativos, informando sua confirmação. Os nós menores atualizam seus registros internos de quem é o líder, e o novo coordenador passa a disparar a rotina de Heartbeats.

4. Ambiente de execução e testes:

Para garantir a corretude da solução e a ausência de condições de corrida ou deadlocks, a validação do sistema foi estruturada em três abordagens complementares:

4.1 Testes Unitários Automatizados: A lógica interna de cada componente foi validada isoladamente através da suíte nativa de testes do Go, o *go test*. Um exemplo de destaque são os testes do Relógio Lógico de Lamport (*lamport_test.go*), que incluem o caso *TestLamportClock_Concurrent*. Neste cenário, dezenas de *goroutines* são criadas simulando eventos de concorrência massiva no acesso ao relógio. Isso garantiu que o uso da primitiva *sync.Mutex* estava, de fato, protegendo a integridade da variável contra corrupção de memória. Além disso, foram criados testes unitários semelhantes para garantir que as trocas de estado isoladas do algoritmo Bully e do Ricart-Agrawala ocorressem conforme o esperado.

4.2 Testes de Integração End-to-End (E2E): Para validar a comunicação de rede de forma programática, desenvolvemos um teste End-to-End (*e2e_test.go*). Este script de teste automatiza o ciclo completo do ambiente:

- Ele orquestra a inicialização de todos os contêineres disparando o comando *docker-compose up --build -d*
- O teste suspende sua execução por 15 segundos, permitindo que a rede estabilize, os nós realizem eleições, sincronizem seus relógios e disputem a seção crítica na prática.
- Em seguida, ele captura a saída completa do comando *docker-compose logs* e realiza afirmações na saída. O teste só passa com sucesso se conseguir detectar,

nos logs gerados, evidências sólidas de que: O relógio de Lamport operou sem falhas; A eleição inicial determinou um líder com sucesso; Os processos conseguiram requisitar e entrar na Seção Crítica provando a exclusão mútua.

4.3 Execução e Simulação de Falhas no Terminal: Além da validação automatizada, o ambiente foi projetado para fácil depuração visual no terminal. Ao executar *docker-compose up --build* na raiz do projeto, os logs de todos os nós são multiplexados na tela, prefixados pelas cores e IDs dos respectivos contêineres, exibindo em tempo real o incremento constante dos timestamps cada nó, as negociações de REQUESTS e REPLY do Ricart e os HEARBEATS do líder a cada 2 segundos.

Por fim, para testar a resiliência do Algoritmo Bully, o principal teste manual realizado consiste em iniciar o cluster com 5 nós ,sendo o Nó 5 o líder natural, e intervir diretamente via terminal executando *docker stop node5*. Ao provocar a queda do líder, pode-se observar pelo terminal o estourar do timeout de 6s nos outros contêineres e a rápida eleição do Nó 4 (o maior ID remanescente), que imediatamente transmite a mensagem COORDINATOR e assume a geração de Heartbeats, sem que o processamento da seção crítica dos outros nós seja afetado.

```
node1 | 12:39:41.428747 [Node 1][Clock: 69] ELECTION | Node 5 inacessível
node4 | 12:39:41.429158 [Node 4][Clock: 69] ELECTION | Node 5 inacessível
node2 | 12:39:41.428353 [Node 2][Clock: 69] ELECTION | Node 5 inacessível
node3 | 12:39:41.430190 [Node 3][Clock: 72] ELECTION | Node 5 inacessível
node4 | 12:39:41.429189 [Node 4][Clock: 70] ELECTION | *** Sou o novo LIDER! *** Enviando COORDINATOR para todos
node2 | 12:39:41.428389 [Node 2][Clock: 69] ELECTION | Recebeu OK, eleicao em andamento em outro no...

node3 | 12:39:41.430253 [Node 3][Clock: 72] ELECTION | Recebeu OK, eleicao em andamento em outro no...
node1 | 12:39:41.428818 [Node 1][Clock: 69] ELECTION | Recebeu OK, eleicao em andamento em outro no...
node2 | 12:39:41.430560 [Node 2][Clock: 75] ELECTION | Novo lider: Node 4 (anterior: Node 5)

node1 | 12:39:41.430466 [Node 1][Clock: 72] ELECTION | Novo lider: Node 4 (anterior: Node 5)
node3 | 12:39:41.430755 [Node 3][Clock: 73] ELECTION | Novo lider: Node 4 (anterior: Node 5)
```

Nó 4 assumindo a liderança

5. Conclusão

Assim, a elaboração deste trabalho prático permitiu solidificar os conceitos teóricos da disciplina de Sistemas Distribuídos. A implementação do zero na linguagem Go ,abandonando frameworks de alto nível, exigiu a construção de uma sistema de comunicação em TCP e nos forçou a lidar diretamente com os desafios de ambientes distribuídos, como assincronia, ausência de estado global e imprevisibilidade da rede.

O desenvolvimento de algoritmos (Relógio Lógico de Lamport para ordenação temporal, Ricart-Agrawala para exclusão mútua e Bully para eleição de coordenador) foi bem-sucedido e validado tanto de forma unitária quanto sistêmica (E2E) através de contêineres Docker. No entanto, o processo revelou dificuldades importantes e limitações aos modelos escolhidos, merecendo destaque:

A Questão do Deadlock: Durante o desenvolvimento do controle de concorrência com o algoritmo de Ricart-Agrawala, o manuseio das travas de rede provou ser um grande desafio. Inicialmente, se um nó bloqueasse a thread principal aguardando a chegada dos REPLYs dos demais participantes, e simultaneamente outro nó fizesse o mesmo, o sistema poderia entrar em deadlock devido à espera circular de respostas no protocolo TCP.

O Desafio da Escalabilidade: Ao analisarmos o volume de tráfego gerado nos logs dos testes, ficou evidente o gargalo de escalabilidade de algoritmos de exclusão mútua totalmente descentralizados. O algoritmo de Ricart-Agrawala requer o envio de $N-1$ mensagens de REQUEST e o recebimento de $N-1$ mensagens de REPLY (onde N é o número de nós) a cada vez que um processo tenta acessar a seção crítica. No nosso experimento com poucos nós, a performance foi satisfatória; porém, fica claro que em um sistema com milhares de processos, a saturação da rede (broadcast storm) tornaria a solução inviável.

Em resumo, a experiência de coordenar nós autônomos para que ajam de maneira sincronizada e recuperem-se automaticamente de falhas (como demonstrado pelo algoritmo Bully) foi extremamente enriquecedora. Ela ilustrou claramente os compromissos que arquitetos devem fazer entre resiliência, corretude e sobrecarga de comunicação ao projetar soluções de grande escala.

6. Referências:

1. Go Documentation - Effective Go. https://go.dev/doc/effective_go
2. Documentação Oficial do Docker e Docker Compose. Disponível em: <https://docs.docker.com/>.
3. TANENBAUM, Andrew S.; VAN STEEN, Maarten. *Sistemas Distribuídos: Princípios e Paradigmas*. 2. ed. São Paulo: Prentice Hall, 2007.